A Layered UML of Bitcoin Primitives, Sub-Protocols, and Extended Kapnet Overlay Layers

Abstract

This document re-expresses the prior UML-style decomposition of Bitcoin as a LaTeX artifact and extends it with a Kapnet-aware superstructure. The goal is not merely to list components, but to organize Bitcoin as a layered system of primitives, commitments, relay rules, execution surfaces, and higher-order protocols. On top of that base, the document introduces an extended family of overlay constructs including AggWit-oriented accounting, weakwork claims, Amethyst Control Tips (ACTs), knots, braids, TXXM messages, multi-base-chain anchoring, and merge-mined or checkpointed coordination surfaces. The result is a readable, systems-level map that treats Bitcoin as a foundational settlement and validation substrate while making room for subordinate cypher-economic coordination overlays.

1 Interpretive Frame

Bitcoin can be modeled as a vertically layered machine whose lower strata define conserved validity and whose upper strata define optional coordination, compression, policy, and application semantics.

At the base are mathematical invariants: hash functions, digital signatures, Merkle commitments, and proof-of-work. Above that sit the consensus-native objects: blocks, transactions, UTXOs, and chainstate transitions. Above those lie script and spending predicates, then encodings, key hierarchies, wallet policy, relay and validation behavior, mining and issuance, deployment mechanisms, and finally higher-level protocols such as Lightning, DLCs, and various anchoring systems.

A Kapnet-style overlay does not replace Bitcoin's base validity rules. Rather, it treats the chain as a deterministic commitment substrate over which richer off-chain or semi-on-chain coordination may be replayed, checkpointed, compressed, and economically priced. In that sense, Bitcoin remains the base physics; Kapnet-like layers become structured semantic metabolism above it.

2 Top-Level Layer Stack

Lyr	Name	Role
0	Mathematical / Cryptographic Foundations	Hashes, elliptic curve signatures, Merkle commitments, proof-of-work, retarget logic, timelock arithmetic.

1	Consensus-Native Ledger Primitives	Headers, blocks, transactions, inputs, outputs, UTXOs, mempool, chain tip, chainstate objects.
2	Script / Spending Layer	Script VM, stack machine semantics, opcode families, standard output types, SegWit, Taproot, Tapscript.
3	Encoding / Commitment Surface	Base58Check, Bech32, Bech32m, WIF, x-only keys, descriptors, Miniscript, derivation notation.
4	Wallet / Signing / Policy Layer	Seeds, HD derivation, accounts, PSBT, signers, coin selection, change policy, watch-only logic.
5	Validation / Relay Layer	Consensus rules, standardness policy, mempool policy, block validation, tx validation, reorg handling, fee estimation.
6	P2P Networking Layer	Peer discovery, relay messages, header-first sync, compact blocks, encrypted transport, anonymity transports.
7	Mining / Issuance / Time Layer	Block assembly, coinbase commitments, subsidy, fee capture, halving, target search, pools and Stratum.
8	Deployment / Chain Evolution Layer	Soft forks, hard forks, activation mechanisms, versionbits, buried deployments, user-driven activation dynamics.
9	Canonical Bitcoin Sub-Protocols	Standardized BIPs such as P2SH, HD wallets, SegWit, Bech32, PSBT, Schnorr, Taproot, Tapscript, BIP324 transport.
10	Bitcoin-Native Higher Protocols	Lightning, DLCs, CoinJoin, PayJoin, Fedimint, Ark, statechains, sidechain concepts, covenant research.
11	Overlay / Application Layer	Timestamping, proof systems, token overlays, Ordinals, inscriptions, Runes, RGB, anchoring and application commitments.
12	Kapnet / Meta-Consensus Overlay Layer	Weakwork, ACTs, knots, braids, TXXM, overlay routing, checkpoint accumulators, multi-tip and multi-base-chain coordination.

3 Bitcoin Core Primitives by Layer

3.1 Layer 0: Mathematical / Cryptographic Foundations

- **SHA-256 / Double-SHA-256**: block header hashing, txid-related hashing surfaces, Merkle node hashing.
- **RIPEMD-160**: address-related hash composition in legacy formats.
- **secp256k1**: elliptic curve group underlying ECDSA and Schnorr.
- **ECDSA**: legacy signature scheme.
- **Schnorr**: linear signature scheme used by Taproot.
- **Merkle Trees**: succinct commitment to transactions and script branches.
- **Hash Preimage / Collision Resistance Assumptions**: security basis of commitments.
- **Proof-of-Work**: cost-imposing ordering mechanism.

- **Difficulty Retarget**: periodic normalization of work threshold.
- **Timelock Arithmetic**: absolute and relative sequencing constraints.

3.2 Layer 1: Consensus-Native Ledger Primitives

- **BlockHeader**: version, previous block hash, Merkle root, time, nBits, nonce.
- **Block**: a header plus ordered transactions and optional witness commitment structures.
- **Transaction**: versioned state transition spending prior outputs and creating new outputs.
- **TxInput**: prevout reference, unlocking material, sequence, witness data where applicable.
- **TxOutput**: value and locking script.
- **OutPoint**: txid plus output index.
- **UTXO**: spendable output residing in chainstate.
- **Coinbase Transaction**: miner-created issuance and fee capture transaction.
- **Mempool**: locally held valid-but-unconfirmed candidate transaction set.
- **ChainTip**: currently selected best valid block frontier.

3.3 Layer 2: Script / Spending Condition Layer

- **Script VM**: stack-based validation environment.
- **Opcodes**: arithmetic, stack, logic, hash, signature and control primitives.
- **P2PK, P2PKH, P2SH, Bare Multisig**: classical spend templates.
- **OP_RETURN**: provably unspendable data commitment surface.
- **CLTV / CSV**: absolute and relative timelocks.
- **SegWit v0**: witness separation, digest changes, malleability reduction.
- **Taproot**: key aggregation-friendly output form.
- **Tapscript**: Taproot script path execution rules.
- **Key-path Spend / Script-path Spend**: dual validation modes under Taproot.
- **MAST / Control Blocks / Annex**: branch commitment and reveal machinery.

3.4 Layer 3: Encodings and Declarative Commitment Surfaces

- **Base58Check**: legacy address and WIF encoding.
- **Bech32 / Bech32m**: SegWit and Taproot-friendly address encodings.
- **WIF**: private key interchange.
- **x-only Public Keys**: Taproot key form.
- **Extended Keys (xpub/xprv family)**: HD wallet derivation objects.
- **BIP32 Paths**: deterministic derivation naming.
- **Descriptors**: machine-readable spending templates.
- **Miniscript**: policy-safe script composition language.

3.5 Layer 4: Wallet / Policy / Signing Layer

- **Wallet / HD Wallet / Seed Material**
- **Account Structures**
- **Coin Selection**
- **Change Output Strategy**
- **PSBT**
- **Signer / Hardware Wallet**
- **Watch-only Wallet**
- **Descriptor Wallet**
- **Spending Policy**

3.6 Layer 5: Validation / Relay Layer

- **Consensus Rules**: validity rules every full validator must enforce.
- **Standardness Rules**: local relay/mining policy beyond consensus.
- **Tx Validation / Block Validation**
- **UTXO Set / Chainstate**
- **Reorg Handling**
- **Mempool Policy**
- **Fee Estimation**

3.7 Layer 6: P2P Networking Layer

- **Peer Objects and Handshake**
- **Version Negotiation**
- **Peer Discovery / DNS Seeds**
- **addr, inv, getdata, tx, block, headers relay families**
- **Header-first Sync**
- **Compact Blocks**
- **BIP324-style Encrypted Transport**
- **Tor / I2P Routing**

3.8 Layer 7: Mining / Issuance / Time Layer

- **Miner**
- **Mining Template**
- **Block Assembler**
- **Coinbase Commitments**
- **Subsidy / Fees**
- **Halving Logic**
- **Difficulty Epoch**
- **Target / Nonce Search / ExtraNonce**
- **Pool / Stratum**

3.9 Layer 8: Deployment / Chain Evolution Layer

- **Genesis / Best Chain / Fork**
- **Soft Fork / Hard Fork**
- **Activation State Machines**
- **Versionbits**
- **Buried Deployments**
- **User-Activated Coordination**

3.10 Layer 9: Canonical Bitcoin Sub-Protocols

Representative standardized modules include:

- BIP16 P2SH
- BIP32 HD wallets
- BIP39 mnemonic seeds
- BIP65 CLTV
- BIP68/BIP112 relative timelock family
- BIP141 SegWit
- BIP143 SegWit digest redesign
- BIP157/158 compact block filters
- BIP173 Bech32
- BIP174 PSBT
- BIP340 Schnorr
- BIP341 Taproot
- BIP342 Tapscript
- BIP324 encrypted transport

3.11 Layer 10: Bitcoin-Native Higher Protocols

- **Lightning**: payment channels, commitment transactions, HTLC/PTLC constructions, watchtowers, factories.
- **DLCs**: oracle-conditioned contract execution.
- **CoinJoin / PayJoin**: transaction construction protocols for privacy and efficiency.
- **Fedimint**: federated Chaumian ecash atop Bitcoin settlement.
- **Ark / Statechains / Sidechain Concepts**: alternative custody and transfer surfaces.
- **Covenant Research**: script-level restrictions for advanced transaction graphs.

3.12 Layer 11: Overlay / Application Layer

- Timestamping
- Proof-of-reserves commitments
- Colored coin style overlays
- Ordinals / inscriptions
- Runes
- RGB
- Nostr anchoring
- External rollup anchors

4 Kapnet-Aware Extension Layer

4.1 Motivation

A Kapnet-aware extension treats Bitcoin blocks and transactions not only as settlement events but also as anchoring and pricing surfaces for a richer coordination fabric. This overlay can remain subordinate to base consensus while introducing additional object types, replay rules, digest checkpoints, weakwork competitions, routing semantics, and data-heavy accumulators.

The overlay therefore separates two questions:

1. What is *Bitcoin-valid*?
2. What is *overlay-valid*, replayable, checkpointable, or rewardable given Bitcoin-valid anchors and a deterministic overlay interpreter?

4.2 Kapnet-Specific Layer 12 Objects

Kapnet Object	**Interpretation**
Weakwork Claim	A low-cost, replayable, deterministic claim object whose admissibility and payoff are governed by overlay rules rather than by base-chain consensus.
ACT (Amethyst Control Tip)	A deterministic digest of overlay state associated with a base-chain progress point, often advancing in blockstep with one or more anchored chains.
Knot	A locally meaningful overlay commitment or claim bundle, potentially representing grouped messages, validations, or routing checkpoints.
Braid	A structured aggregate of multiple knots, claims, or message flows whose relationships matter more than any single linear chain.

TXXM	Transaction CrossMail, a generalized message-transaction form extending the expressive surface of Bitcoin-like settlement events.
Overlay Accumulator	A commitment structure summarizing a large set of overlay events, claims, or state transitions.
Checkpoint Commitment	A digest committed on-chain or cross-chain that binds replayable overlay history to a succinct root.
Weakwork Lottery Surface	Deterministic winner-selection logic over candidate claims, often fee-aware, timing-aware, or policy-aware.
AggWit Accounting Surface	An extended weight or entropy accounting layer used for assigning internal burdens, rewards, or sanitation metrics above base validity.
Merge-Mined Overlay Header	An auxiliary or nested header describing overlay work or claims synchronized with a base chain or another difficulty-adjusted chain.
Provider Nodule	A node, service, or DAO-level operator responsible for routing, indexing, policy, or weakwork market functionality.
Guild / Room / Submission Object	Social-coordination primitives represented as overlay-native state objects replayable by deterministic interpreters.

4.3 Kapnet Sub-Layers

It is useful to internally decompose Layer 12 into six sub-layers.

12A. Overlay Message Objects

These include TXXMs, submission packets, validation packets, governance packets, routing hints, and cross-domain references.

12B. Overlay Execution and Validation

These include KScript-like deterministic interpreters, bounded replay semantics, instruction/resource accounting, domain and room constraints, and dependency checks.

12C. Overlay State Commitment

These include ACT digests, accumulator roots, knot roots, braid summaries, and checkpoint proofs.

12D. Overlay Economics

These include burn, bond, postage, stake, change splits; weakwork pricing; reputation-aware routing; and lottery settlement surfaces.

12E. Overlay Networking

These include provider nodules, relay meshes, domain-specific transport, peer routing, persistence guarantees, and checkpoint dissemination.

12F. Multi-Base-Chain Coordination

These include BTC-anchored overlays, auxiliary chains, merge-mined side systems, ACT synchronization across multiple chaintips, and chain-specific weakwork namespaces.

5 UML-Style Dependency Description

The following longtable is a textual UML decomposition. It states which object depends on, contains, or commits to which other object.

Object	Depends on / Contains / Commits to
Double-SHA256	SHA-256
Digital Signature	secp256k1
Schnorr / ECDSA	secp256k1
Merkle Tree	Hash function family
Proof-of-Work	Double-SHA256, target comparison, retarget schedule
Block	Block header, ordered transactions, optional witness commitment
Transaction	Inputs, outputs, version, locktime
TxInput	Outpoint, scriptSig, sequence, optional witness
TxOutput	Value, scriptPubKey
UTXO	TxOutput plus unspent status in chainstate
CoinbaseTx	Transaction, subsidy rules, fee aggregation
Script VM	Script, stack, opcode semantics
P2PKH / P2SH / Taproot Output	ScriptPubKey template families
SegWit v0	Witness separation, digest redesign
Taproot	x-only key, Schnorr, script tree commitments
Descriptor	Script form, key origin metadata, policy description
PSBT	Unsigned transaction, metadata, signer coordination fields
Wallet	Key material, descriptors, coin selection, change logic
Full Node	Consensus rules, tx validation, block validation, chainstate
Mempool	Valid unconfirmed transactions under local policy
Peer	Transport session, negotiated features, relay messages
Mining Template	Candidate tx set, coinbase data, policy-selected ordering
Coinbase Commitment Surface	Miner-chosen commitments, issuance, fees, optional auxiliary digests
Soft Fork	New consensus restriction layered over prior valid set
Lightning Channel	Funding transaction, commitment transactions, contract clauses
DLC	Oracle signatures, contract settlement logic, Bitcoin transactions
Ordinals / Inscriptions	Transaction ordering logic, witness data carriage
RGB	Bitcoin commitment points, client-side validation
Weakwork Claim	Overlay message object, deterministic validity predicates, optional base anchor
ACT	Overlay accumulator roots, knot/braid summaries, chain progress point

Knot	Set of overlay events or claims with local structural relation
Braid	Ordered or partially ordered family of knots/claims
TXXM	Transaction-like message containing routing, action, validation, and payload fields
AggWit Accounting	Tx/block data, extended weight semantics, entropy or delta attribution logic
Merge-Mined Overlay Header	Base chain progress point, auxiliary header fields, overlay proof logic
Provider Nodule	Relay role, indexing role, policy role, economic role
Guild / Room / Submission	Overlay state objects interpreted by replayable governance or discourse rules

6 Formal Layer Relations

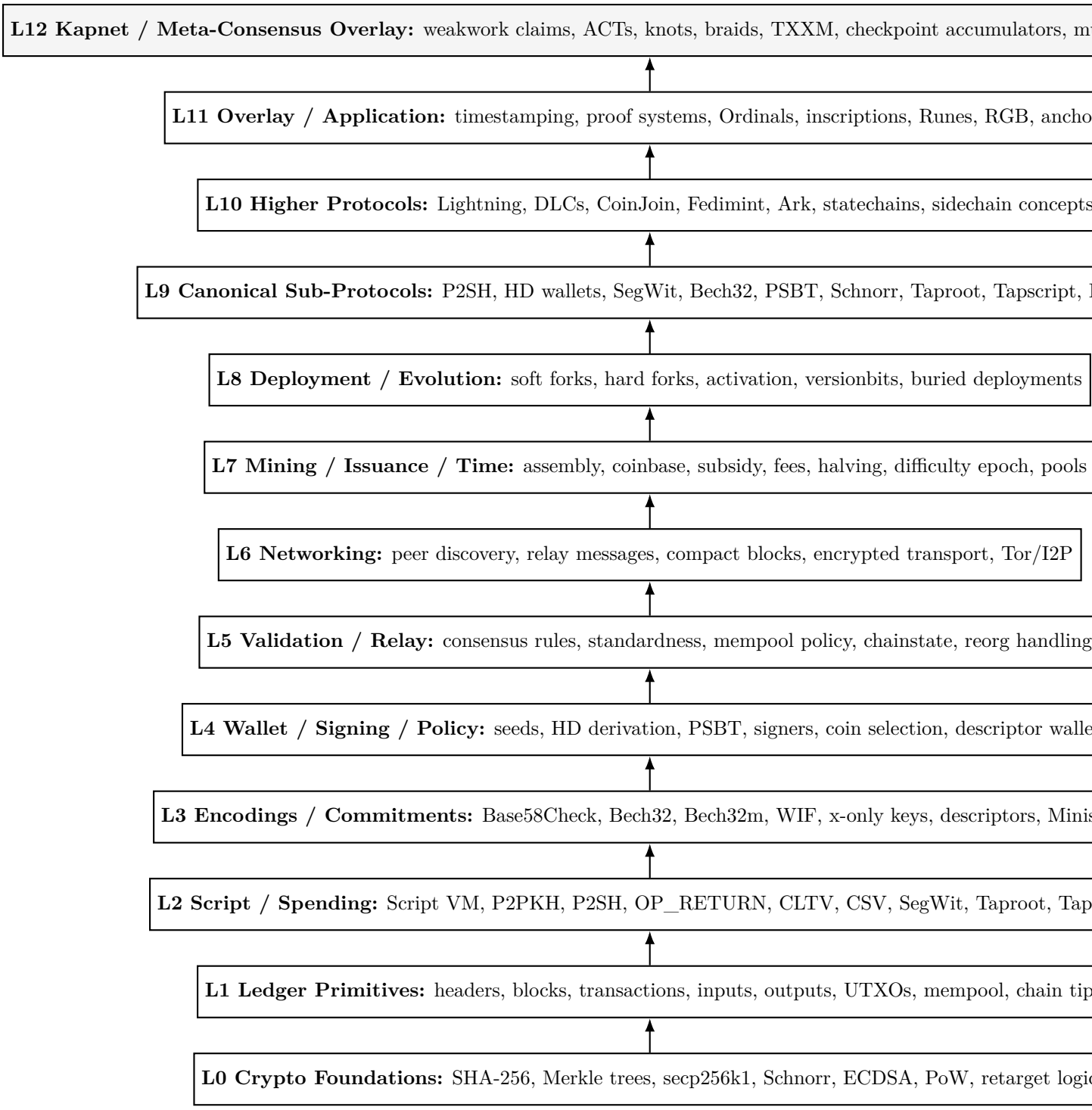
We can summarize the architecture as a containment and commitment chain:

$$L_0 \to L_1 \to L_2 \to L_3 \to L_4 \to L_5 \to L_6 \to L_7 \to L_8 \to L_9 \to L_{10} \to L_{11} \to L_{12}$$

where the arrow means "provides the validity or commitment substrate for."

Bitcoin-native validity lives primarily across L_0 through L_8. Standardized and application-level expressive systems live across L_9 through L_{11}. Kapnet-like meta-consensus and semantic coordination systems live at L_{12}, subordinate to but expressive over the lower layers.

7 TikZ UML-Style Diagram



8 Kapnet-Aware Extended Relations

Below is the compact extension the earlier UML needed.

Base-Chain Anchoring Relations

- A **base block** may commit, directly or indirectly, to overlay-relevant data.
- A **transaction** may carry or reference overlay payloads via OP_RETURN, witness data, script-committed structures, or external indexable commitments.
- A **coinbase commitment surface** may carry merge-mined or auxiliary overlay digests.
- A **chain tip** provides the discrete clock against which ACT advancement may be defined.

Overlay State Relations

- A **TXXM** may commit to actions, validation claims, governance changes, routing instructions, or subordinate payload references.
- A set of TXXMs may be grouped into a **knot**.
- Multiple knots with structural relation may form a **braid**.
- A braid, together with auxiliary replay metadata, may be summarized into an **overlay accumulator**.
- An overlay accumulator root may be included in an **ACT**.
- An ACT may be synchronized in blockstep with one or more base chains, or with multi-tip families where each maintains its own digest lineage.

Economic Relations

- A TXXM or overlay packet may include a **burn-bond-postage-stake-change** decomposition.
- A **weakwork claim** may compete in a deterministic overlay lottery.
- A **provider nodule** may accept fees, relay packets, or offer specialized weakwork and routing services.
- **AggWit-style accounting** may re-attribute block or transaction burden for overlay-internal pricing or sanitation without changing base consensus unless adopted at base level.

Multi-Base-Chain Relations

- Distinct base chains may each have a local ACT lineage.
- Overlay packets may specify chain-qualified namespaces such as BTC:kap:claim, BTC:kap:BCK:claim, or further nested route identifiers.
- Merge-mined auxiliaries may expose additional blockspace markets while remaining anchored to a heavier settlement chain.
- Replay semantics may require deterministic cross-chain ordering or checkpoint windows.

9 A Compact UML Taxonomy

Bitcoin Base Classes

1. `HashPrimitive`
2. `SignaturePrimitive`
3. `CommitmentPrimitive`
4. `ProofOfWorkPrimitive`
5. `LedgerObject`
6. `ScriptObject`
7. `EncodingObject`
8. `WalletObject`
9. `ValidationObject`
10. `NetworkObject`
11. `MiningObject`
12. `DeploymentObject`
13. `ProtocolExtension`

Kapnet Extension Classes

1. `OverlayMessage`
2. `OverlayExecutionRule`
3. `OverlayCheckpoint`
4. `OverlayEconomicObject`
5. `OverlayTopologyObject`
6. `OverlayGovernanceObject`
7. `OverlayWeakworkObject`
8. `MultiChainCoordinationObject`

10 Why This Layering Matters

This architecture clarifies several important distinctions.

First, not all meaningful Bitcoin-adjacent behavior is consensus behavior. Many useful systems live above consensus while still depending on the chain as their final commitment substrate.

Second, sub-protocols differ in how deeply they penetrate the stack. Some, like Taproot, modify the script and signature substrate. Others, like PSBT, mostly operate at the wallet coordination layer. Others still, like Lightning or Fedimint, inhabit higher layers while relying on lower settlement guarantees.

Third, a Kapnet-style overlay is best understood not as a replacement blockchain but as a structured meta-consensus fabric. It can be replayable, deterministic, economically expressive, and even multi-tip-aware without requiring every semantic distinction to be enforced by base consensus.

11 Conclusion

Bitcoin can be rendered as a layered UML-style machine whose lower levels secure validity and whose upper levels enable increasingly rich coordination. A Kapnet-aware extension adds a superstructure of weakwork, accumulator checkpoints, ACTs, knots, braids, TXXMs, and multi-base-chain replay domains. This preserves the separation between base-chain validity and overlay semantics while giving a formal place to structured cypher-economic coordination.

In practical terms, the diagram says:

> Bitcoin provides conserved validity, scarce settlement, and globally replayable anchoring.
>
> Kapnet-like overlays provide higher-order routing, coordination, pricing, governance, and semantic state transitions over that base.